

Rapport : Affectation des étudiants en Master

Implémentation de Gale-Shapley et résolution par programmation linéaire.

Jesus Alejandro Gomez Urzua
Nadhir Ben Said

7 mai 2026

Table des matières

1	Introduction	1
2	Algorithme de Gale-Shapley	1
2.1	Optimisation avec des structures de données	2
2.2	Adaptation pour l'exécution côté parcours	2
2.3	Jeux de tests et analyse de performances	2

1 Introduction

Ce projet porte sur l'étude et la résolution du problème de l'affectation des étudiants au sein des différents parcours de Master. L'objectif est de coupler n étudiants à m formations en tenant compte de leurs préférences respectives, tout en respectant les capacités d'accueil limitées propres à chaque Master. L'enjeu majeur réside dans l'implémentation d'algorithmes capables de satisfaire simultanément plusieurs critères :

1. Stabilité : garantir qu'aucun couple (étudiant, Master) n'ait intérêt à rompre l'affectation actuelle pour se choisir mutuellement.
2. Efficacité : maximiser le bien-être total des participants selon une approche utilitariste.
3. Équité : assurer un niveau de satisfaction minimal pour tous les agents suivant une approche égalitariste.

Pour ce faire, nous proposons dans un premier temps une implémentation de l'algorithme de Gale-Shapley, qui garantit l'obtention d'un mariage stable sans toutefois assurer l'efficacité ou l'équité de la solution. Dans un second temps, nous utiliserons la programmation linéaire afin d'optimiser les critères d'efficacité et d'équité, indépendamment des contraintes de stabilité.

2 Algorithme de Gale-Shapley

L'algorithme de Gale-Shapley repose sur une stratégie de type « propositions et rejets » qui garantit l'obtention d'un mariage stable. La version implémentée dans ce projet adapte le modèle vu en cours pour intégrer les capacités d'accueil des Masters. Dans cette configuration, un parcours accepte une proposition tant qu'il dispose de places vacantes. Une fois sa capacité atteinte, il ne valide une nouvelle candidature que si l'étudiant proposant est mieux classé que l'étudiant le moins préféré parmi ceux déjà acceptés.

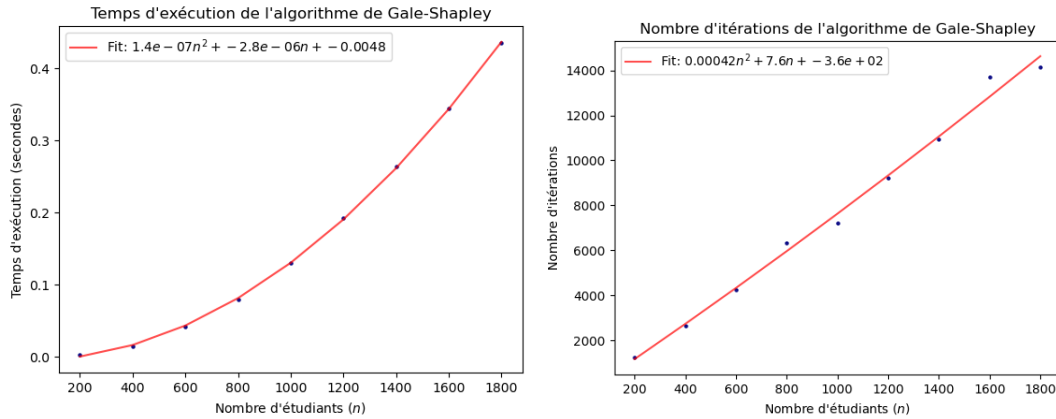


FIGURE 1 – Exécution de l'algorithme Gale-Shapley pour des données aléatoires.

2.1 Optimisation avec des structures de données

Afin de garantir une complexité efficace, nous avons couplé chaque opération élémentaire de l'algorithme à une structure de données adaptée :

1. **Identification des étudiants libres** : L'utilisation d'une `deque` permet d'extraire un étudiant et de le réinsérer en cas de rejet en $\mathcal{O}(1)$.
2. **Sélection du prochain parcours** : Un tableau d'entiers mémorise pour chaque étudiant l'indice du prochain Master à solliciter dans sa liste de préférences, garantissant une sélection en $\mathcal{O}(1)$.
3. **Évaluation du rang** : Le pré-calcul d'une matrice des rangs en $\mathcal{O}(n \times m)$ permet à chaque Master de connaître en temps constant la position d'un candidat dans son classement.
4. **Gestion des admissions** : Chaque Master gère ses étudiants admis via un tas binaire. Cette structure permet d'identifier l'étudiant le moins préféré en $\mathcal{O}(1)$ et de mettre à jour l'affectation en $\mathcal{O}(\log(k))$, où k est la capacité maximale d'un parcours.

Ainsi, notre implémentation atteint une complexité globale de $\mathcal{O}(n \times m \log(k))$

2.2 Adaptation pour l'exécution côté parcours

Il est important de noter que cet algorithme n'est pas symétrique : il favorise systématiquement le groupe qui émet les propositions. En l'occurrence, cette version est *étudiant-optimale*, offrant à chaque étudiant la meilleure affectation possible parmi tous les mariages stables, tout en étant *parcours-pessimale*.

Dans la version où les Masters proposent aux étudiants, l'implémentation se rapproche davantage du modèle abordé en cours et devient plus simple. En effet, la gestion des priorités par un tas binaire devient superflue : chaque étudiant ne pouvant être affecté qu'à un seul Master, il lui suffit de comparer toute nouvelle offre à son affectation actuelle pour l'accepter ou la rejeter. Pour gérer les capacités d'accueil, chaque Master est inséré dans la `deque` autant de fois qu'il possède de places disponibles. Ainsi, la somme des capacités étant égale au nombre d'étudiants, dans cette version la complexité de $\mathcal{O}(n^2)$.

2.3 Jeux de tests et analyse de performances

La fiabilité de l'implémentation a été vérifiée par un jeu de tests via la bibliothèque `unittest`. Divers scénarios d'affectation ont été élaborés manuellement et stockés dans un fichier JSON incluant

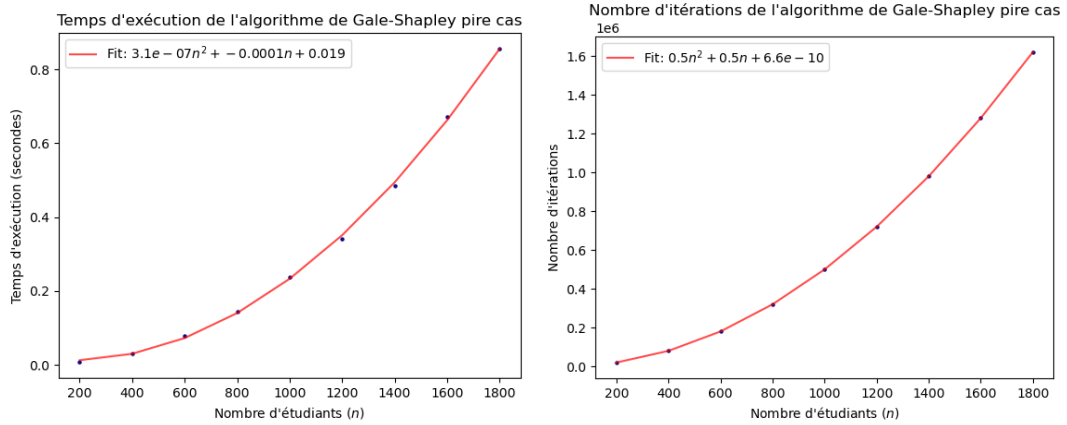


FIGURE 2 – Exécution de l'algorithme Gale-Shapley avec des données forçant le pire cas.

les résultats attendus. Ces tests couvrent l'algorithme de Gale-Shapley (variantes étudiant et parcours) ainsi que la fonction de détection des paires instables.

Pour confronter les performances observées à la complexité théorique, nous avons mesuré le temps d'exécution et le nombre d'itérations pour des instances allant de 200 à 2000 agents.

Dans un premier temps, l'algorithme a été testé sur des matrices de préférences aléatoires et uniformes. Les résultats dans la Figure 1 montrent un temps d'exécution quadratique, alors que le nombre d'itérations suit une croissance plus lente. Cette observation est cohérente avec les travaux de Wilson (1972), qui démontrent que le nombre moyen de propositions est en $\mathcal{O}(n \log n)$ sous l'hypothèse de préférences indépendantes et uniformes. La divergence entre le temps et les itérations s'explique par la phase de pré-calcul, dont la complexité est en $\mathcal{O}(n \times m)$ quel que soit le contenu des préférences.

Dans un second temps, nous avons soumis l'implémentation à un scénario de pire cas (Figure 2). Dans cette configuration, le temps de calcul et le nombre d'itérations suivent tous deux un comportement quadratique, confirmant ainsi la borne supérieure théorique de l'algorithme.

Références

Wilson, L. B. (1972), 'An analysis of the stable marriage assignment algorithm', *BIT Numerical Mathematics* **12**(4), 569–575.